

Package ‘LongFPLS’

May 2, 2026

Type Package

Title Longitudinal Functional Partial Least Squares Regression

Version 1.0.0

Description Functions for fitting longitudinal functional partial least squares regression models for longitudinal scalar-on-function data. The package includes tools for simulation-based data generation, pilot decomposition of longitudinal functional predictors into subject-specific functional intercept, subject-specific functional slope, and visit-specific deviation components, residualized supervised partial least squares construction, REML-based mixed-model fitting, coefficient-function reconstruction, smoothing, and BIC-based tuning of LFPLS components.

Depends R (>= 3.5.0)

Imports MASS,
nlme,
splines,
stats

License GPL-3

Encoding UTF-8

LazyData true

ByteCompile true

RoxygenNote 7.1.2

Contents

lfpls_iterate	2
prepare_lfpls	5
simulate_data	9
tune_lfpls	12

Index	17
--------------	-----------

lfpls_iterate

*Fit Longitudinal Functional Partial Least Squares***Description**

Fits the longitudinal functional partial least squares (LFPLS) model for a fixed choice of tuning parameters. The function takes an object produced by [prepare_lfpls](#), extracts supervised component-specific PLS directions, refits the score mixed model by REML, and reconstructs the component-specific coefficient functions.

Usage

```
lfpls_iterate(prepare,
              n_comp_b0 = 3,
              n_comp_b1 = 3,
              n_comp_u = 3,
              ortho = TRUE,
              ridge = 1e-10,
              lambda_b = 1e-2,
              max_iter = 8,
              tol = 1e-4,
              lme_maxiter = 80)
```

Arguments

prepare	An object returned by prepare_lfpls . It contains the scalar response, subject identifiers, standardized visit times, scalar covariate design matrix, numerical integration weights, initial variance-component estimates, and the estimated longitudinal functional decomposition.
n_comp_b0	A positive integer giving the number of LFPLS components extracted from the between-subject functional intercept component $B_{i0}(s)$. The default is 3.
n_comp_b1	A positive integer giving the number of LFPLS components extracted from the between-subject functional slope component $t_{ij}B_{i1}(s)$. The default is 3.
n_comp_u	A positive integer giving the number of LFPLS components extracted from the visit-specific deviation component $U_{ij}(s)$. The default is 3.
ortho	Logical. If TRUE, the Krylov-type PLS basis vectors are orthonormalized before constructing the score variables. The default is TRUE.
ridge	A small nonnegative ridge constant used to stabilize matrix inversions during residualization against scalar covariates and previously extracted score components. The default is 1e-10.
lambda_b	Nonnegative smoothing parameter used in the discrete Sobolev-type regularization of the reconstructed coefficient functions. Larger values produce smoother coefficient estimates. The default is 1e-2.
max_iter	Maximum number of outer LFPLS iterations. Each iteration updates the whitened PLS directions, the REML mixed-model fit, the variance components, and the reconstructed coefficient functions. The default is 8.
tol	Convergence tolerance for the weighted squared change in the reconstructed coefficient functions. The default is 1e-4.
lme_maxiter	Maximum number of iterations passed to the lme optimizer when fitting the score mixed model. The default is 80.

Details

The function implements the fixed-tuning LFPLS fitting step for longitudinal scalar-on-function regression. The underlying working model is based on the decomposition

$$X_{ij}(s) = \eta(s, t_{ij}) + B_{i0}(s) + t_{ij}B_{i1}(s) + U_{ij}(s),$$

where $\eta(s, t)$ is a smooth mean surface, $B_{i0}(s)$ and $B_{i1}(s)$ are subject-specific functional intercept and slope processes, and $U_{ij}(s)$ is a visit-specific deviation process.

After preprocessing by [prepare_lfpls](#), LFPLS fits the component-wise mixed model

$$Y_{ij} = \gamma^\top z(t_{ij}) + \theta^\top Z_{ij} + \langle B_{i0}, \beta_0 \rangle + \tilde{t}_{ij} \langle B_{i1}, \beta_1 \rangle + \langle U_{ij}, \beta_U \rangle + b_i + \epsilon_{ij}.$$

The inferential target is the triple of component-specific coefficient functions $(\beta_0, \beta_1, \beta_U)$.

At each outer iteration, the function first forms the stacked functional design based on the weighted empirical components B_{i0} , $\tilde{t}_{ij}B_{i1}$, and U_{ij} . It then whitens the response, scalar covariates, and functional blocks using the current random-intercept covariance estimates. For subject i , the working covariance has the form

$$V_i = \sigma_e^2 I_{J_i} + \sigma_b^2 \mathbf{1}_{J_i} \mathbf{1}_{J_i}^\top.$$

This whitening step accounts for within-subject correlation when extracting the supervised functional directions.

For the between-subject blocks, residualized alternative partial least squares (RAPLS) directions are constructed after projecting both the response and functional blocks away from the scalar covariate design. The directions are generated through a Krylov-type recursion based on the residualized empirical covariance and cross-covariance operators. In the discretized implementation, this is equivalent to repeatedly applying cross-products of the residualized functional design to the residualized response.

The visit-specific block U_{ij} is treated sequentially. Its PLS directions are extracted after residualizing against both the scalar covariates and the already extracted between-subject score variables. This prevents the within-subject component from re-capturing predictive signal already explained by the subject-level intercept and slope components.

Given the extracted score variables, the function fits a linear mixed model with a scalar random intercept by REML using `lme`. The fixed effects associated with the LFPLS score variables are then used to reconstruct the coefficient functions:

$$\hat{\beta}_0(s) = \sum_{\ell=1}^{p_0} \hat{\delta}_{0\ell} \psi_\ell^{(0)}(s), \quad \hat{\beta}_1(s) = \sum_{\ell=1}^{p_1} \hat{\delta}_{1\ell} \psi_\ell^{(1)}(s), \quad \hat{\beta}_U(s) = \sum_{\ell=1}^{p_U} \hat{\delta}_{U\ell} \psi_\ell^{(U)}(s).$$

The raw reconstructed coefficient vectors are smoothed using a discrete second-difference penalty, corresponding to a Sobolev-type roughness regularization. A diminishing-step averaging update is used to stabilize successive coefficient-function estimates.

This function is intended for fitting LFPLS with fixed values of `n_comp_b0`, `n_comp_b1`, `n_comp_u`, and `lambda_b`. For automatic tuning over grids of these parameters, use [tune_lfpls](#).

Value

A list containing the fitted LFPLS model and associated estimates:

<code>fit</code>	The fitted <code>lme</code> object from the final REML score mixed model.
<code>b0</code>	Numeric vector containing the estimated coefficient function $\hat{\beta}_0(s)$ for the between-subject functional intercept component, evaluated on the functional grid used in prepare_lfpls .

b1	Numeric vector containing the estimated coefficient function $\hat{\beta}_1(s)$ for the between-subject functional slope component, evaluated on the functional grid used in prepare_lfpls .
bU	Numeric vector containing the estimated coefficient function $\hat{\beta}_U(s)$ for the visit-specific deviation component, evaluated on the functional grid used in prepare_lfpls .
n_comp_b0	Number of LFPLS components used for the B_{i0} block.
n_comp_b1	Number of LFPLS components used for the $t_{ij}B_{i1}$ block.
n_comp_u	Number of LFPLS components used for the U_{ij} block.
lambda_b	Smoothing parameter used for the coefficient-function regularization.
iter	Number of outer LFPLS iterations completed.
w	Numerical integration weights over the functional grid.
sigma_e	Final REML estimate of the residual standard deviation.
sigma_b	Final REML estimate of the random-intercept standard deviation.

If no score mixed model can be fitted successfully, the returned object may be NULL.

Note

The input object `prep` must be created by [prepare_lfpls](#). The estimated coefficient vectors `b0`, `b1`, and `bU` are returned on the same functional grid used to construct the input functional predictor matrix. The function uses a scalar random-intercept mixed model for the response-side dependence structure.

See Also

[prepare_lfpls](#), [tune_lfpls](#), [simulate_data](#)

Examples

```
## Simulate a small longitudinal scalar-on-function dataset
dat <- simulate_data(
  case = "1",
  sigma_x = 0.05,
  sigma_y = 0.35,
  sigma_b = 0.50
)

X <- dat$X
y <- dat$y
Z <- dat$Z
subject <- dat$subject
times <- dat$times
tps <- dat$tps

## Prepare the LFPLS input object
prep <- prepare_lfpls(
  y = y,
  subject = subject,
  times = times,
  X = X,
  tps = tps,
```

```

    scalar_cov = Z,
    presmooth_X = TRUE,
    df_spline = 10,
    df_time_eta = 5,
    df_time_Z = 4,
    lme_maxiter = 80
  )

  ## Fit LFPLS with fixed tuning parameters
  fit <- lfpls_iterate(
    prep = prep,
    n_comp_b0 = 1,
    n_comp_b1 = 1,
    n_comp_u = 1,
    lambda_b = 1e-2,
    max_iter = 2
  )

  names(fit)
  fit$iter
  fit$sigma_e
  fit$sigma_b

  ## Estimated component-specific coefficient functions
  plot(dat$tps, fit$b0, type = "l",
       xlab = "Functional domain",
       ylab = "Coefficient",
       main = "Estimated beta_0")

```

prepare_lfpls

Prepare Data for Longitudinal Functional PLS

Description

Constructs the prepared data object required for longitudinal functional partial least squares (LF-PLS). The function estimates the pilot longitudinal functional decomposition, builds the scalar and time-effect design matrices, computes numerical integration weights, fits an initial random-intercept mixed model, and stores all quantities required by [lfpls_iterate](#) and [tune_lfpls](#).

Usage

```

prepare_lfpls(y, subject, times, X, tps,
             scalar_cov = NULL,
             presmooth_X = TRUE,
             df_spline = 10,
             df_time_eta = 5,
             df_time_Z = 4,
             lme_maxiter = 80)

```

Arguments

y	A numeric vector of scalar responses. Its length must be equal to the number of rows of X.
----------	--

subject	A vector of subject identifiers. Observations with the same identifier are treated as repeated visits from the same subject.
times	A numeric vector of visit times. The function internally standardizes these times before estimating the longitudinal functional decomposition and constructing the time-effect basis.
X	A numeric matrix containing the observed functional predictor. Rows correspond to subject-visit observations and columns correspond to grid points over the functional domain.
tps	A numeric vector giving the grid points of the functional domain. Its length must be equal to <code>ncol(X)</code> . The vector is used for optional presmoothing and for constructing numerical integration weights.
scalar_cov	Optional scalar covariates. This may be NULL, a numeric vector, a numeric matrix, or a data frame. If a data frame is supplied, it is expanded using <code>model.matrix(~. - 1, data = scalar_cov)</code> . The default is NULL.
presmooth_X	Logical. If TRUE, each observed functional predictor curve is presmoothed over tps using <code>smooth.spline</code> . The default is TRUE.
df_spline	Degrees of freedom used in <code>smooth.spline</code> when <code>presmooth_X = TRUE</code> . The default is 10.
df_time_eta	Degrees of freedom for the spline basis used to estimate the smooth mean surface $\eta(s, t)$ in the pilot longitudinal functional decomposition. The default is 5.
df_time_Z	Degrees of freedom for the spline basis used to model the smooth fixed effect of visit time in the scalar response model. The default is 4.
lme_maxiter	Maximum number of iterations passed to the lme optimizer when fitting the initial random-intercept mixed model. The default is 80.

Details

The function prepares the data required to fit LFPLS for longitudinal scalar-on-function regression. The method assumes that, for subject i and visit j , the observed functional predictor can be represented through the longitudinal decomposition

$$X_{ij}(s) = \eta(s, t_{ij}) + B_{i0}(s) + t_{ij}B_{i1}(s) + U_{ij}(s),$$

where $\eta(s, t)$ is a smooth population mean surface, $B_{i0}(s)$ is a subject-specific functional intercept process, $B_{i1}(s)$ is a subject-specific functional slope process, and $U_{ij}(s)$ is a visit-specific functional deviation process.

The preparation step proceeds as follows. First, the response, subject identifiers, visit times, and functional predictor matrix are coerced to the required formats, and the function checks that `nrow(X)` equals `length(y)`. If `presmooth_X = TRUE`, each curve is smoothed over the functional grid tps.

Second, visit times are standardized and a smooth mean surface $\eta(s, t)$ is estimated using a spline basis in the visit-time direction. The centered curves are then decomposed subject by subject into empirical estimates of the functional intercept $B_{i0}(s)$, functional slope $B_{i1}(s)$, and visit-specific deviation $U_{ij}(s)$.

Third, numerical integration weights are computed over tps using the trapezoidal rule. These weights are used to construct the weighted functional blocks required by the LFPLS score construction: B_{i0} , $\tilde{t}_{ij}B_{i1}$, and U_{ij} . In the returned object these are stored as `B0_wtd`, `B1t_wtd`, and `U_wtd`. The weighted blocks approximate the functional inner products used in the scalar-on-function model.

Fourth, the scalar fixed-effect design is constructed. A spline basis in standardized visit time is included to represent the smooth population-level time effect. Optional scalar covariates supplied through `scalar_cov` are appended to this time-basis design. The resulting matrix is stored as `Z_full`.

Finally, an initial random-intercept mixed model is fitted by REML:

$$Y_{ij} = \gamma^\top z(t_{ij}) + \theta^\top Z_{ij} + b_i + \epsilon_{ij},$$

where b_i is a subject-specific scalar random intercept. The resulting initial estimates of the residual and random-intercept standard deviations are stored as `sigma_e0` and `sigma_b0`. These estimates are used to initialize the whitening step in `lfpls_iterate`.

The returned object is not itself a final LFPLS fit. It is a structured input object for `lfpls_iterate`, which fits LFPLS for fixed tuning parameters, and `tune_lfpls`, which searches over tuning parameter grids.

Value

A list with the following components:

<code>y</code>	Numeric vector of scalar responses.
<code>subject</code>	Vector of subject identifiers.
<code>subject_factor</code>	Factor version of <code>subject</code> , used in mixed-model fitting.
<code>n</code>	Total number of subject-visit observations.
<code>D</code>	Number of grid points in the functional predictor.
<code>tstd</code>	Standardized visit times.
<code>w</code>	Numerical integration weights over the functional grid, computed using the trapezoidal rule.
<code>Z_full</code>	Full scalar fixed-effect design matrix, including the spline basis for standardized visit time and any additional scalar covariates.
<code>z_names</code>	Column names of <code>Z_full</code> .
<code>df0</code>	Data frame used to fit the initial reduced mixed model.
<code>formula0</code>	Formula object for the initial reduced mixed model.
<code>fit0</code>	Initial lme fit of the reduced random-intercept mixed model.
<code>sigma_e0</code>	Initial REML estimate of the residual standard deviation.
<code>sigma_b0</code>	Initial REML estimate of the scalar random-intercept standard deviation.
<code>B0_wtd</code>	Weighted functional block corresponding to the subject-specific functional intercept component $B_{i0}(s)$.
<code>B1t_wtd</code>	Weighted functional block corresponding to the standardized-time-weighted functional slope component $\tilde{t}_{ij} B_{i1}(s)$.
<code>U_wtd</code>	Weighted functional block corresponding to the visit-specific deviation component $U_{ij}(s)$.
<code>penalty</code>	Discrete second-difference penalty matrix used later to smooth the reconstructed coefficient functions.
<code>decomposition</code>	A list containing the pilot functional decomposition, including the presmoothed/used functional predictor matrix, standardized times, estimated mean surface, centered curves, subject-specific functional intercepts, subject-specific functional slopes, and visit-specific deviations.

Note

The object returned by `prepare_lfpls` is designed to be passed directly to `lfpls_iterate` or `tune_lfpls`. The functional predictor matrix X should be observed on a common dense grid `tps`. If `presmooth_X = TRUE`, `tps` must be supplied and must have length equal to `ncol(X)`.

See Also

[lfpls_iterate](#), [tune_lfpls](#), [simulate_data](#)

Examples

```
## Simulate a small longitudinal scalar-on-function dataset
dat <- simulate_data(
  case = "1",
  sigma_x = 0.05,
  sigma_y = 0.35,
  sigma_b = 0.50
)

X <- dat$X
y <- dat$y
Z <- dat$Z
subject <- dat$subject
times <- dat$times
tps <- dat$tps

## Prepare the LFPLS input object
prep <- prepare_lfpls(
  y = y,
  subject = subject,
  times = times,
  X = X,
  tps = tps,
  scalar_cov = Z,
  presmooth_X = TRUE,
  df_spline = 10,
  df_time_eta = 5,
  df_time_Z = 4,
  lme_maxiter = 80
)

names(prepare)
prep$n
prep$D

## Initial variance-component estimates
prep$sigma_e0
prep$sigma_b0

## Weighted functional blocks used by LFPLS
dim(prepare$B0_wtd)
dim(prepare$B1t_wtd)
dim(prepare$U_wtd)

## Fit LFPLS using the prepared object
```



```

fit <- lfpls_iterate(
  prep = prep,
  n_comp_b0 = 1,
  n_comp_b1 = 1,
  n_comp_u = 1,
  lambda_b = 1e-2,
  max_iter = 2
)

names(fit)

```

simulate_data

*Simulate Longitudinal Scalar-on-Function Data***Description**

Generates simulated longitudinal scalar-on-function data for evaluating longitudinal functional partial least squares (LFPLS). The function creates longitudinal functional predictors with subject-specific functional intercept and slope components, visit-specific functional deviations, scalar covariates, random intercepts, and scalar responses.

Usage

```

simulate_data(case = c("1", "2"),
              nI = 100,
              nV = 4,
              tps = seq(0.5, 99.5, by = 1),
              sigma_x = 0.05,
              sigma_y = 0.35,
              sigma_b = 0.50,
              q_scalar = 5,
              theta_true = NULL)

```

Arguments

case	Character string specifying the simulation scenario. Use "1" for the global functional-effect case and "2" for the component-specific longitudinal functional-effect case.
nI	Number of subjects. The default is 100.
nV	Nominal number of visits per subject. The function generates an unbalanced longitudinal design with total sample size approximately $nI * nV$. The default is 4.
tps	Numeric vector giving the grid points of the functional domain. The default is <code>seq(0.5, 99.5, by = 1)</code> .
sigma_x	Standard deviation of the additive measurement noise in the observed functional predictor. The default is 0.05.
sigma_y	Standard deviation of the scalar response error. The default is 0.35.
sigma_b	Standard deviation of the scalar subject-specific random intercept in the response model. The default is 0.50.

q_scalar	Number of scalar covariates generated for the scalar part of the response model. If q_scalar = 0, no scalar covariates are included. The default is 5.
theta_true	Optional numeric vector of true scalar regression coefficients. If NULL, the default value is c(0.6, -0.4, 0.3, 0.2, 0.1). Its length should be compatible with q_scalar.

Details

The function generates data from a longitudinal scalar-on-function simulation design motivated by the LFPLS model. For subject i and visit j , the latent functional predictor is generated from the longitudinal decomposition

$$X_{ij}^{latent}(s) = \eta(s, t_{ij}) + B_{i0}(s) + t_{ij}B_{i1}(s) + U_{ij}(s),$$

where $\eta(s, t)$ is a smooth mean surface, $B_{i0}(s)$ is a subject-specific functional intercept component, $B_{i1}(s)$ is a subject-specific functional slope component, and $U_{ij}(s)$ is a visit-specific deviation component.

The functional components are generated using a weighted orthonormal basis constructed from cubic B-spline functions, trigonometric functions, and localized Gaussian-shaped functions over the standardized functional domain. The coefficient vectors for the subject-specific and visit-specific functional components are generated from multivariate normal distributions with Toeplitz covariance structures. The observed functional predictor is then generated by adding independent Gaussian measurement noise:

$$X_{ij}(s) = X_{ij}^{latent}(s) + e_{ij}^X(s).$$

The scalar response includes a scalar random intercept, optional scalar covariate effects, and a functional regression signal. Specifically, case = "1" corresponds to a global-effect setting in which the response is driven by a single coefficient function acting on the full latent functional predictor:

$$Y_{ij} = b_i + Z_{ij}^\top \theta + 1.50 \int X_{ij}^{latent}(s) \beta(s) ds + \epsilon_{ij}.$$

This case represents a setting where a single global coefficient function is sufficient to describe the functional regression effect.

In contrast, case = "2" corresponds to a component-specific longitudinal setting in which the response depends separately on the between-subject functional component and the visit-specific deviation component:

$$Y_{ij} = b_i + Z_{ij}^\top \theta + 1.60 \int B_{ij}^{comp}(s) \beta_B(s) ds + 1.10 \int U_{ij}^{comp}(s) \beta_U(s) ds + \epsilon_{ij}.$$

This second case is designed to evaluate methods that exploit the longitudinal decomposition of the functional predictor. It is particularly relevant for comparing supervised LFPLS with unsupervised longitudinal functional principal component regression and global penalized functional regression approaches.

The function returns both the observed data and the latent quantities used to generate them. Therefore, it is useful for simulation studies, method comparison, debugging, and examples for [prepare_lfpls](#), [lfpls_iterate](#), and [tune_lfpls](#).

Value

A list containing the simulated data and truth:

case	Simulation case used to generate the data.
------	--

nI	Number of subjects.
n	Total number of subject-visit observations.
Jvec	Vector containing the number of visits for each subject.
subject	Subject identifier for each observation.
times	Standardized visit times for all subject-visit observations.
tps	Functional grid points.
X	Observed functional predictor matrix. Rows correspond to subject-visit observations and columns correspond to functional grid points.
y	Simulated scalar response vector.
mu_true	Conditional mean of the scalar response before adding the response error.
Z	Matrix of generated scalar covariates. If q_scalar = 0, this is a matrix with zero columns.
theta_true	True scalar regression coefficient vector used to generate the scalar covariate effect.
W_latent	Latent functional predictor matrix before additive measurement noise is added.
B_comp	Between-subject functional component used in the response-generating mechanism.
U_comp	Visit-specific functional deviation component used in the response-generating mechanism.
eta_mat	Smooth mean-surface component evaluated at the observed visit times and functional grid points.
Psi	Weighted orthonormal basis used to generate the functional components.
truth	A list containing the true coefficient functions. For case = "1", truth\$beta_true contains the global coefficient function and truth\$beta_B_true and truth\$beta_U_true are NULL. For case = "2", truth\$beta_B_true and truth\$beta_U_true contain the true between-subject and within-subject coefficient functions, while truth\$beta_true is NULL.

See Also

[prepare_lfpls](#), [lfpls_iterate](#), [tune_lfpls](#)

Examples

```
## Case 1: global functional coefficient setting
dat1 <- simulate_data(
  case = "1",
  sigma_x = 0.05,
  sigma_y = 0.35,
  sigma_b = 0.50
)

names(dat1)
dim(dat1$X)
length(dat1$y)
table(dat1$subject)
```

```

## True global coefficient function
plot(dat1$tps, dat1$truth$beta_true, type = "l",
      xlab = "Functional domain",
      ylab = "Coefficient",
      main = "True global coefficient function")

## Case 2: component-specific longitudinal effects
dat2 <- simulate_data(
  case = "2",
  sigma_x = 0.05,
  sigma_y = 0.35,
  sigma_b = 0.50
)

## True between-subject and within-subject coefficient functions
plot(dat2$tps, dat2$truth$beta_B_true, type = "l",
      xlab = "Functional domain",
      ylab = "Coefficient",
      ylim = range(dat2$truth$beta_B_true, dat2$truth$beta_U_true),
      main = "True between-subject coefficient")

lines(dat2$tps, dat2$truth$beta_U_true, lty = 2)

## Prepare the simulated data for LFPLS
prep <- prepare_lfpls(
  y = dat2$y,
  subject = dat2$subject,
  times = dat2$times,
  X = dat2$X,
  tps = dat2$tps,
  scalar_cov = dat2$Z,
  presmooth_X = TRUE,
  df_spline = 10,
  df_time_eta = 5,
  df_time_Z = 4,
  lme_maxiter = 80
)

fit <- tune_lfpls(
  prep = prep,
  n_comp_b0_grid = 1:5,
  n_comp_b1_grid = 1:5,
  n_comp_u_grid = 1:5,
  lambda_b_grid = c(1e-2, 5e-2, 1e-1),
  max_iter = 6,
  tol = 1e-4,
  ortho = TRUE,
  ridge = 1e-10,
  lme_maxiter = 80
)

names(fit)

```

Description

Selects the tuning parameters of longitudinal functional partial least squares (LFPLS) by grid search. The function fits LFPLS over candidate numbers of component-specific PLS directions and smoothing parameters, then returns the model with the smallest Bayesian information criterion (BIC).

Usage

```
tune_lfpls(prepare,
           n_comp_b0_grid = 1:5,
           n_comp_b1_grid = 1:5,
           n_comp_u_grid = 1:5,
           lambda_b_grid = c(1e-3, 1e-2),
           max_iter = 6,
           tol = 1e-4,
           ortho = TRUE,
           ridge = 1e-10,
           lme_maxiter = 80)
```

Arguments

prepare	An object returned by prepare_lfpls . It contains the prepared longitudinal functional decomposition, scalar design matrix, integration weights, initial variance-component estimates, and all quantities required to fit LFPLS.
n_comp_b0_grid	Integer vector giving the candidate numbers of LFPLS components extracted from the between-subject functional intercept component $B_{i0}(s)$. The default is 1:5.
n_comp_b1_grid	Integer vector giving the candidate numbers of LFPLS components extracted from the between-subject functional slope component $\tilde{t}_{ij}B_{i1}(s)$. The default is 1:5.
n_comp_u_grid	Integer vector giving the candidate numbers of LFPLS components extracted from the visit-specific deviation component $U_{ij}(s)$. The default is 1:5.
lambda_b_grid	Numeric vector of candidate nonnegative smoothing parameters for the component-specific coefficient functions. The default is c(1e-3, 1e-2).
max_iter	Maximum number of outer LFPLS iterations used in each candidate fit. The default is 6.
tol	Convergence tolerance passed to lfpls_iterate . The default is 1e-4.
ortho	Logical. If TRUE, the Krylov-type PLS basis vectors are orthonormalized when constructing the LFPLS directions. The default is TRUE.
ridge	Small nonnegative ridge constant used to stabilize residualization and matrix inversions inside lfpls_iterate . The default is 1e-10.
lme_maxiter	Maximum number of iterations passed to the lme optimizer for each candidate score mixed model. The default is 80.

Details

The function performs automatic tuning for the LFPLS estimator. For each candidate tuple

$$(p_0, p_1, p_U, \lambda) \in \mathcal{P}_0 \times \mathcal{P}_1 \times \mathcal{P}_U \times \Lambda,$$

where $\mathcal{P}_0$, $\mathcal{P}_1$, and $\mathcal{P}_U$ are the supplied component-number grids and Λ is the supplied smoothing-parameter grid, `tune_lfpls` calls `lfpls_iterate`.

For a fixed candidate tuple, `lfpls_iterate` fits the LFPLS working model

$$Y_{ij} = \gamma^\top z(t_{ij}) + \theta^\top Z_{ij} + \langle B_{i0}, \beta_0 \rangle + \tilde{t}_{ij} \langle B_{i1}, \beta_1 \rangle + \langle U_{ij}, \beta_U \rangle + b_i + \epsilon_{ij},$$

where B_{i0} , B_{i1} , and U_{ij} are obtained from the pilot longitudinal functional decomposition stored in `prep`. The parameters p_0 , p_1 , and p_U determine the dimensions of the LFPLS Krylov subspaces used for the between-subject intercept, between-subject slope, and visit-specific deviation blocks, respectively. The parameter λ controls the Sobolev-type smoothing of the reconstructed coefficient functions.

Each candidate fit produces a REML score mixed model. The BIC of this mixed model is computed using `BIC(fit$fit)`. Candidate fits that fail, return `NULL`, or do not produce a finite BIC are skipped. The selected model is the successful candidate with the smallest finite BIC:

$$(\hat{p}_0, \hat{p}_1, \hat{p}_U, \hat{\lambda}) = \arg \min_{(p_0, p_1, p_U, \lambda)} \text{BIC}\{\widehat{\mathcal{M}}(p_0, p_1, p_U, \lambda)\}.$$

The selected BIC value is stored in the returned object as `best_bic`.

This function is useful when the analyst does not want to manually choose the number of LFPLS components for each longitudinal functional block or the smoothing strength for the reconstructed coefficient functions. For large grids or high-resolution functional predictors, the search may be computationally intensive because `lfpls_iterate` is refitted for every candidate combination.

Value

A fitted LFPLS object corresponding to the candidate tuning parameters with the smallest finite BIC. The returned object is the list produced by `lfpls_iterate`, with an additional component:

<code>fit</code>	The fitted lme object from the selected REML score mixed model.
<code>b0</code>	Estimated coefficient function $\hat{\beta}_0(s)$ for the between-subject functional intercept component.
<code>b1</code>	Estimated coefficient function $\hat{\beta}_1(s)$ for the between-subject functional slope component.
<code>bU</code>	Estimated coefficient function $\hat{\beta}_U(s)$ for the visit-specific functional deviation component.
<code>n_comp_b0</code>	Selected number of LFPLS components for the B_{i0} block.
<code>n_comp_b1</code>	Selected number of LFPLS components for the $\tilde{t}_{ij}B_{i1}$ block.
<code>n_comp_u</code>	Selected number of LFPLS components for the U_{ij} block.
<code>lambda_b</code>	Selected smoothing parameter.
<code>iter</code>	Number of outer LFPLS iterations completed for the selected fit.
<code>w</code>	Numerical integration weights over the functional grid.
<code>sigma_e</code>	Final REML estimate of the residual standard deviation for the selected fit.
<code>sigma_b</code>	Final REML estimate of the scalar random-intercept standard deviation for the selected fit.
<code>best_bic</code>	Smallest finite BIC value found over the tuning grid.

If all candidate fits fail, the function prints the last error message, when available, and stops with an error.

Note

The object supplied to `prep` must first be created by `prepare_lfpls`. The total number of candidate fits equals `length(n_comp_b0_grid) * length(n_comp_b1_grid) * length(n_comp_u_grid) * length(lambda_b_grid)`. For computationally expensive applications, start with a small grid and then refine the grid near the best candidate.

See Also

[prepare_lfpls](#), [lfpls_iterate](#), [simulate_data](#)

Examples

```
## Simulate a small longitudinal scalar-on-function dataset
dat <- simulate_data(
  case = "2",
  sigma_x = 0.05,
  sigma_y = 0.35,
  sigma_b = 0.50
)

X <- dat$X
y <- dat$y
Z <- dat$Z
subject <- dat$subject
times <- dat$times
tps <- dat$tps

## Prepare the LFPLS input object
prep <- prepare_lfpls(
  y = y,
  subject = subject,
  times = times,
  X = X,
  tps = tps,
  scalar_cov = Z,
  presmooth_X = TRUE,
  df_spline = 10,
  df_time_eta = 5,
  df_time_Z = 4,
  lme_maxiter = 80
)

## Tune LFPLS over a small grid
fit_tuned <- tune_lfpls(
  prep = prep,
  n_comp_b0_grid = 1:5,
  n_comp_b1_grid = 1:5,
  n_comp_u_grid = 1:5,
  lambda_b_grid = c(1e-2, 5e-2, 1e-1),
  max_iter = 6,
  tol = 1e-4,
  ortho = TRUE,
  ridge = 1e-10,
  lme_maxiter = 80
)
```

```
)

## Selected tuning parameters
fit_tuned$n_comp_b0
fit_tuned$n_comp_b1
fit_tuned$n_comp_u
fit_tuned$lambda_b
fit_tuned$best_bic

## Estimated component-specific coefficient functions
plot(dat$tps, fit_tuned$b0, type = "l",
      xlab = "Functional domain",
      ylab = "Coefficient",
      main = "Tuned LFPLS: beta_0")
```


Index

- * **functional partial least squares**

- tune_lfpls, [12](#)

- * **longitudinal functional data**

- tune_lfpls, [12](#)

- * **model selection**

- tune_lfpls, [12](#)

- * **models**

- tune_lfpls, [12](#)

- * **regression**

- tune_lfpls, [12](#)

lfpls_iterate, [2](#), [5](#), [7](#), [8](#), [10](#), [11](#), [13–15](#)

prepare_lfpls, [2–4](#), [5](#), [10](#), [11](#), [13](#), [15](#)

simulate_data, [4](#), [8](#), [9](#), [15](#)

tune_lfpls, [3–5](#), [7](#), [8](#), [10](#), [11](#), [12](#)